Appunti

Workflow Orchestrator

Simona Selvetti

Abstract—Il presente elaborato analizza l'architettura e le funzionalità del **Workflow Orchestrator** (WFO), focalizzandosi sull'implementazione di riferimento `example-orchestrator`.

1. INTRODUZIONE

Workflow Orchestrator (WFO) è un framework software open source progettato per l'automazione e l'orchestrazione dei processi di rete.

Il suo scopo principale è gestire l'intero ciclo di vita dei servizi (come nodi, link o circuiti virtuali) attraverso l'esecuzione di *workflow* deterministici. WFO agisce come un livello di astrazione intelligente: riceve le intenzioni dell'operatore e le traduce in azioni concrete sui sistemi sottostanti (OSS e dispositivi fisici), garantendo che la configurazione della rete rimanga consistente, tracciabile e allineata con i sistemi di inventario.

2. ARCHITETTURA DEL SISTEMA E STACK TECNOLOGICO

2.1. Docker

Docker è una piattaforma di containerizzazione open-source che consente di impacchettare un'applicazione e tutte le sue dipendenze (librerie, file di configurazione) in un'unità standardizzata e isolata chiamata **container**. Viene utilizzato per garantire la portabilità e la riproducibilità assoluta del software, assicurando che l'ambiente di esecuzione rimanga identico indipendentemente dalla macchina su cui viene avviato, eliminando così i problemi di compatibilità. Nello specifico, in questo progetto Docker gestisce l'orchestrazione simultanea di tutti i servizi necessari all'ecosistema: istanzia e connette in una rete virtuale il core applicativo, l'interfaccia utente, i database di supporto e i sistemi esterni fondamentali, permettendo di avere un ambiente operativo completo con un singolo comando.

2.2. Ochestrator Core

L'Orchestrator Core è il motore applicativo centrale del framework, sviluppato in Python. Viene utilizzato per definire ed eseguire la logica di business tramite macchine a stati finiti, gestendo l'intero ciclo di vita dei prodotti di rete (creazione, modifica, terminazione e validazione). Nello specifico, questo componente è responsabile dell'esecuzione passo-passo dei workflow: riceve gli input, calcola le transizioni di stato, interagisce con i servizi esterni (come NetBox) tramite moduli dedicati e garantisce che ogni operazione sia atomica e tracciabile, persistendo lo stato su database.

2.3. Orchestrator UI

L'Orchestrator UI è l'interfaccia grafica web (frontend), che funge da punto di interazione per gli operatori di rete. Viene utilizzata per avviare nuovi processi, monitorare l'avanzamento dei workflow in tempo reale e gestire i task manuali o falliti. Nello specifico, la sua caratteristica distintiva è la generazione dinamica dei form: l'interfaccia non è "cablata", ma costruisce automaticamente le maschere di input (campi di testo, menu a tendina, validazioni) basandosi sugli schemi dati (JSON Schema/Pydantic) inviati dal backend, permettendo agli sviluppatori di creare nuovi prodotti lato codice senza dover mai modificare il codice del frontend.

2.4. NetBox

NetBox è una piattaforma open-source di Infrastructure Resource Modeling (IRM) che combina funzionalità di IP Address Management (IPAM) e Datacenter Infrastructure Management (DCIM). Viene utilizzato per fungere da "Source of Truth" (SoT) autoritativa per l'intera infrastruttura di rete, centralizzando la documentazione sia degli asset fisici (dispositivi, rack, cavi) che delle risorse logiche (indirizzi IP, VLAN, prefissi) in un unico database strutturato. Nello specifico, all'interno di questo progetto, NetBox opera come sistema di inventario (IMS) dinamico: l'orchestratore interagisce direttamente con le sue API (REST e GraphQL) per creare, aggiornare o validare automaticamente gli oggetti di rete , assicurando che ogni operazione di provisioning si rifletta istantaneamente e accuratamente nella documentazione dell'infrastruttura.

2.5. PostgreSQL

PostgreSQL è un sistema di gestione di database relazionale a oggetti (ORDBMS) open-source, estremamente robusto e performante. Viene utilizzato per garantire la persistenza sicura e strutturata di tutti i dati critici gestiti dall'applicazione, fungendo da memoria a lungo termine per l'intero ecosistema software. Nello specifico, all'interno di questo progetto, agisce come backend di archiviazione primario sia per l'Orchestrator che per NetBox; esso memorizza i dettagli di tutte le sottoscrizioni ai prodotti (come le configurazioni dei nodi e dei link), gestisce le migrazioni degli schemi dati tramite Alembic e salva lo stato serializzato di ogni workflow passo dopo passo, permettendo al sistema di mantenere traccia dell'avanzamento dei processi e di garantire l'integrità dei dati anche in caso di riavvii.

2.6. Redis

Redis è uno store di strutture dati in-memory open-source, noto per la sua elevatissima velocità in quanto conserva i dati nella RAM anziché su disco. Viene utilizzato per fungere da message broker (intermediario di messaggi) e sistema di caching temporaneo, gestendo la comunicazione asincrona tra i diversi componenti software. All'interno di questo progetto è fondamentale per la gestione delle code dei processi: quando un workflow viene avviato, i singoli compiti (task) non vengono eseguiti tutti immediatamente ma vengono inviati a Redis che "li mette in fila". Questo permette all'Orchestrator di gestire operazioni lunghe e complesse in background (come la configurazione remota di apparati) mantenendo l'interfaccia utente sempre reattiva e veloce.

2.7. Federation

Federation (o più specificamente Apollo Federation) è un'architettura per API GraphQL che consente di comporre più servizi backend distinti (chiamati subgraph) in un unico grafo dati unificato, noto come supergraph. Viene utilizzato per fornire ai client un unico punto di accesso (endpoint) per interrogare dati distribuiti su sistemi diversi, eliminando la necessità di effettuare chiamate multiple e incrociare i risultati manualmente lato applicazione. In WFO il servizio di Federation agisce da gateway unificando le API dell'Orchestrator e quelle di NetBox; questo permette, ad esempio, di effettuare una singola query che recupera contemporaneamente lo stato logico di una sottoscrizione (dal database dell'Orchestrator) e i dettagli tecnici fisici del dispositivo associato (dal database di NetBox), come se risiedessero nello stesso sistema.

3. MODELLAZIONE DEL DOMINIO E WORKFLOW

L'architettura logica di WFO si fonda su due pilastri concettuali: la modellazione gerarchica dei prodotti e l'orchestrazione del loro ciclo di vita tramite macchine a stati.

3.1. Gerarchia dei Prodotti

Il framework adotta una struttura che separa nettamente la definizione astratta di un servizio dalla sua realizzazione concreta. Questa modellazione viene implementata tramite classi Python (Domain Models) che utilizzano la libreria *Pydantic* per la validazione dei tipi a runtime.

Le entità principali sono:

- **Product Type**: Rappresenta la definizione di alto livello di ciò che può essere erogato (es. "Link Core", "Porta Utente"). Definisce gli input fissi, come la velocità di una porta, che discriminano le varianti dello stesso prodotto.
- **Product Block**: È l'unità atomica e riutilizzabile della modellazione. I prodotti complessi sono costruiti componendo gerarchicamente più blocchi. Questo approccio modulare riduce la duplicazione del codice e facilita la manutenzione.
- **Subscription**: È l'istanza univoca di un Prodotto, legata a un cliente specifico (Customer). Una sottoscrizione possiede un identificativo univoco (UUID) e contiene i dati di configurazione reali (Resource Types) specifici per quell'istanza (es. Indirizzi IP, VLAN ID).

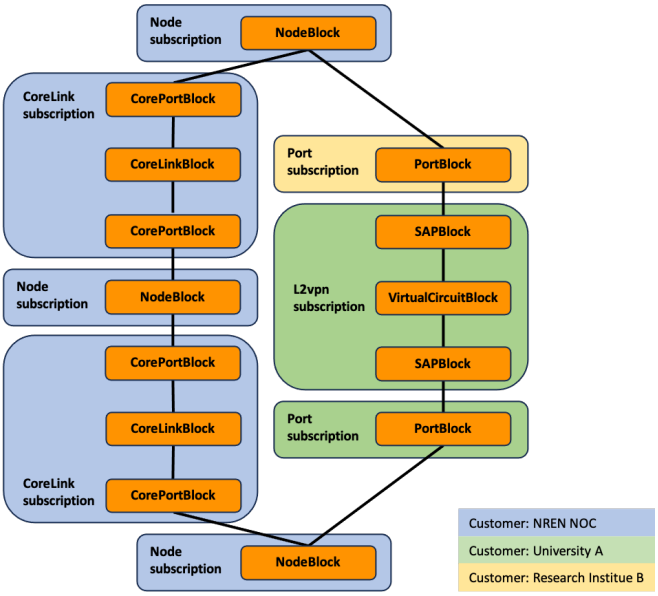

Figure 1. Diagramma della gerarchia dei prodotti

3.2. Gestione del Ciclo di Vita

Una caratteristica distintiva del framework è che le Sottoscrizioni non sono entità statiche, ma evolvono attraverso stati del ciclo di vita predefiniti: *Initial*, *Provisioning*, *Active* e *Terminated*.

Il framework utilizza l'ereditarietà delle classi per applicare regole di validazione diverse a seconda dello stato. Ad esempio, un `NodeBlockInactive` avrà molti campi opzionali, mentre il corrispondente `NodeBlockActive` renderà quegli stessi campi obbligatori, garantendo che una risorsa non possa essere considerata "Attiva" se mancano i dati essenziali.

3.3. Architettura dei Workflow

L'interazione con le sottoscrizioni avviene esclusivamente tramite l'esecuzione di **Workflow**. Questi sono processi deterministici composti da una sequenza di passaggi atomici (*Steps*). Il framework definisce quattro tipologie di workflow fondamentali:

1. **Create**: Gestisce l'inizializzazione della sottoscrizione, raccoglie gli input utente tramite form generati dinamicamente e orchestra il provisioning iniziale sui sistemi esterni.
2. **Modify**: Permette di alterare i parametri di una sottoscrizione esistente. Transiziona lo stato in *Provisioning*, applica le modifiche e, solo in caso di successo, riporta lo stato in *Active*.
3. **Terminate**: Gestisce la dismissione ordinata del servizio, rimuovendo le configurazioni dalla rete e liberando le risorse negli inventari.
4. **Validate**: Un processo speciale che verifica la congruenza tra lo stato registrato nel database dell'orchestrator e lo stato reale dei sistemi esterni, rilevando eventuali disallineamenti (*drift*).

Tecnicamente, il passaggio di informazioni tra uno step e l'altro avviene tramite un oggetto *State* (un dizionario serializzato). Ogni step è una funzione Python idempotente che riceve in input lo stato corrente, esegue un'operazione e restituisce uno stato aggiornato.

4. STRUTTURA DEL CODICE E SVILUPPO

Un aspetto fondamentale dell'`example-orchestrator` è l'adesione a rigorose *Best Common Practices* (BCP) riguardanti l'organizzazione del codice sorgente. Il progetto adotta una struttura modulare progettata per facilitare la navigazione e la scalabilità man mano che nuovi prodotti e workflow vengono aggiunti.

4.1. Organizzazione del File System

Il progetto adotta una struttura di directory standardizzata, progettata per mantenere separati i domini logici e facilitare la navigazione del codice. Di seguito viene riportato l'albero delle directory principali, seguito da un'analisi dettagliata di ciascun componente:

```
|-- migrations
|   |-- versions
|   `-- schema
|-- products
|   |-- product_blocks
|   |-- product_types
|   `-- services
|-- services
|-- templates
|-- translations
|-- utils
`-- workflows
    |-- <product>
    `-- tasks
```

`migrations/` Gestisce il versionamento dello schema del database. Utilizza *Alembic*, uno strumento parte di SQLAlchemy, per tracciare le modifiche strutturali (es. nuove tabelle per nuovi prodotti). La cartella `versions` contiene gli script di migrazione incrementali.

`products/` È il cuore della modellazione del dominio.

- `product_types/`: Contiene i file di definizione dei prodotti di alto livello (es. `node.py`), descrivendone il ciclo di vita e gli input fissi.
- `product_blocks/`: Definisce i componenti riutilizzabili che compongono i prodotti (es. `core_port.py`), permettendo una struttura gerarchica e modulare.
- `services/`: Contiene funzioni helper specifiche per il dominio dei prodotti, come la generazione automatica delle descrizioni delle sottoscrizioni o la costruzione dei payload per le API esterne.

`services/` A differenza della cartella precedente, questa directory nella root ospita le integrazioni trasversali e i client per i sistemi esterni. Qui risiede, ad esempio, la logica di comunicazione con NetBox o altri OSS/BSS, implementata tramite pattern come il *single dispatch* per gestire il polimorfismo delle chiamate.

`templates/` Contiene i template di configurazione utilizzati per generare automaticamente lo scheletro del codice (scaffolding). Questa funzionalità sperimentale permette di creare rapidamente la struttura base per nuovi prodotti, workflow e migrazioni.

`translations/` Gestisce l'internazionalizzazione (i18n) dell'interfaccia utente, contenendo le traduzioni per i campi dei form, le descrizioni dei prodotti e i messaggi dei workflow.

`utils/` Raccoglie funzioni di utilità generale non strettamente legate alla business logic, come script per il setup dell'ambiente di deployment o la generazione di documentazione tecnica.

`workflows/` Contiene la logica procedurale per l'orchestrazione.

- `<product>/`: Ogni prodotto ha una propria sottocartella (es. `port/`) che raggruppa i file per i processi di creazione, modifica e terminazione specifici per quel servizio.
- `tasks/`: Contiene workflow di sistema o di manutenzione che non sono legati a una specifica sottoscrizione utente, come ad esempio il task per il popolamento iniziale (bootstrap) di NetBox.

4.2. Sviluppo dei Modelli e Type Safety

L'implementazione dei modelli fa ampio uso della libreria **Pydantic**. Questo approccio consente di definire le proprietà dei prodotti (come la velocità di una porta o il nome di un nodo) utilizzando le *Type Hints* di Python. Il vantaggio principale di questa metodologia è la **validazione a runtime**: il framework converte automaticamente i dati grezzi provenienti dal database o dagli input utente nei tipi Python corretti, sollevando eccezioni in caso di incongruenze. Ciò garantisce che i workflow operino sempre su dati validati e strutturati, riducendo drasticamente i bug dovuti a dati malformati.

4.3. Implementazione dei Workflow

Dal punto di vista dello sviluppatore, un workflow non è altro che una sequenza di funzioni Python decorate con `@step`. Il framework utilizza un meccanismo di *Dependency Injection* basato sui nomi e sui tipi delle variabili: lo stato del workflow viene passato da uno step all'altro e le variabili necessarie vengono iniettate automaticamente nella funzione dello step successivo. Questo rende il codice estremamente leggibile e testabile, poiché ogni step è una funzione pura o isolata che esegue un singolo compito atomico.

A. CONFIGURAZIONE AMBIENTE TEST

Guida per la configurazione dell'ambiente *example-orchestrator* su sistema operativo Linux.

A.1. Prerequisiti del sistema

Per replicare l'infrastruttura è necessaria la presenza dei seguenti strumenti sulla macchina host:

- **Git**: Per il versionamento e il download del codice sorgente.
- **Docker Engine** (e il plugin **Docker Compose**): Per l'orchestrazione dei container. L'ambiente è compatibile sia con Linux (consigliato) che con Windows/Mac tramite Docker Desktop.

A.2. Acquisizione del Codice Sorgente

```
git clone https://github.com/SimoSelv/example-orchestrator-stable.git
cd example-orchestrator
```

Nota

utilizziamo una versione stabilizzata su un altro repository in quanto quella ufficiale presenta problemi con gli ultimi commit.

- Revert to cf7e8b2 consistent formatting
- fissate le versioni in docker-compose.yml[ln 152/165] (4.7.1 per orchestrator-core e 7.0.0 per example-orchestrator-ui)

A.3. Configurazione delle Variabili d'Ambiente

TODO

file interessati (.../example-orchestrator/docker/):

- netbox.env
- orchestrator.env
- orchestrator-ui.env
- redis.env

configurazione di default

- user: `admin`
- password: `admin`
- **Orchestrator UI**: http://localhost:3000
- **NetBox IMS**: http://localhost:8000
- **Federation**: http://localhost:4000
- **Swagger API**: http://localhost:8080/api/doc (Documentazione interattiva delle API backend).

A.4. Avvio dei Servizi

A.4.1. Avvio standard

L'avvio dell'intero stack applicativo avviene tramite Docker Compose, che si occupa di scaricare le immagini necessarie (pull), creare la rete virtuale interna e montare i volumi per la persistenza dei dati:

```
docker compose up -d
```

L'opzione `-d` (detached) esegue i processi in background. In questa fase vengono inizializzati 5 container principali: *orchestrator*, *orchestrator-ui*, *netbox*, *postgres* e *redis*.

A.4.2. Avvio con Profilo LSO (Opzionale)

TODO

Se si necessita del modulo *Lightweight Service Orchestrator* (LSO) per l'integrazione con Ansible, è necessario passare una variabile d'ambiente specifica prima del comando di avvio, che istruisce Docker a compilare e avviare anche il container LSO:

```
COMPOSE_PROFILES=lso docker compose up -d
```

A.5. Verifica dello Stato dei Servizi

Una volta lanciato il comando di avvio, è fondamentale verificare che tutti i container siano in stato `running` e che le porte siano state mappate correttamente. Eseguire:

```
docker compose ps
```

L'output atteso deve mostrare i seguenti servizi attivi sulle rispettive porte:

- **orchestrator-ui**: porta 3000
- **netbox**: porta 8000.
- **federation**: porta 4000.
- **postgres** e **redis**: porte standard database (interne).

A.6. Inizializzazione (Bootstrap)

Sebbene l'infrastruttura sia avviata da riga di comando, la logica applicativa richiede un'inizializzazione dei dati su NetBox (Source of Truth). Secondo la documentazione ufficiale, questa operazione è gestita tramite un Task asincrono invocabile dall'interfaccia web.

Non essendoci uno script CLI nativo documentato per questa fase specifica nel repository di esempio, procedere come segue per finalizzare la configurazione:

1. Aprire il browser all'indirizzo http://localhost:3000/tasks.
2. Nella sezione `Task` cliccare il bottone `New task`
3. Selezionare dal menù a tendina il task `NetBox Bootstrap`.
4. Cliccare `Start task`

Questo passaggio popola il database con i siti, i ruoli e i prefissi necessari per l'esecuzione dei workflow successivi.

A.7. Terminazione

Per arrestare l'intero stack e rimuovere le risorse di rete create:

```
docker compose down
```